FAST NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES

CS-4063: Natural Language Processing

Assignment 3 — Transformers + RAG

Total Marks: 80 + 5 Bonus
Due Date: 29-04-26, 11:59 PM

Restrictions — Read Before You Start

Both the encoder-only and decoder-only Transformer models must be implemented entirely from scratch. You are not allowed to use any pretrained models such as BERT, GPT, T5, or any library that abstracts away the internal Transformer implementation.The following built ins are **not** allowed: nn.Transformer, nn.MultiheadAttention, nn.TransformerEncoder.

Transformer-based Review Understanding with RAG Enhanced Explanation Generation

In this assignment, you will build a three-stage NLP system that first extracts structured information from text, retrieves relevant examples from the dataset, and then generates grounded natural language explanations. The three stages — encoding, retrieval, and generation — are implemented in Parts A, B, and C respectively, and must work together as a coherent pipeline.

Dataset Description

You will use the Amazon Reviews dataset, available at:

https://nijianmo.github.io/amazon/index.html

You are required to construct your own dataset subset. To ensure diversity, you must select reviews from at least three different product categories, including approximately ten to fifteen thousand reviews per category, resulting in a final dataset of between thirty thousand and forty five thousand samples. Each sample must include the review text and its corresponding star rating on a scale of one to five.

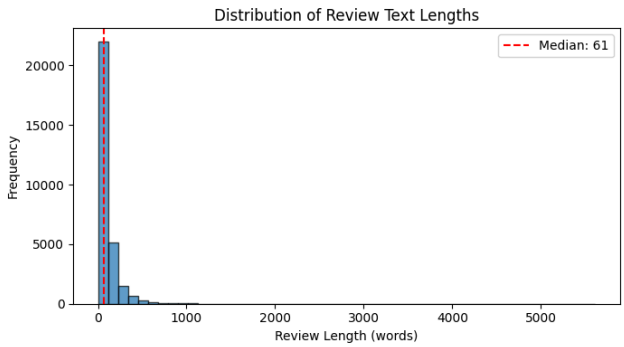
You must divide your dataset into training, validation, and testing sets using a 70%, 15%, and 15% split respectively. You are fully responsible for preparing the dataset and ensuring it is suitable for training.

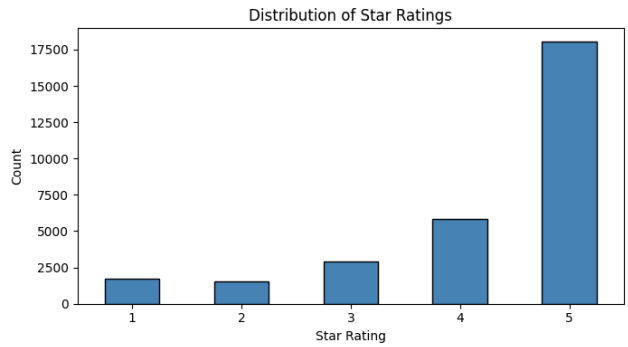
Preprocessing

All preprocessing must be implemented by you using the techniques covered in previous lectures. Your pipeline should include text cleaning where necessary, tokenization, vocabulary construction

using only the training data, conversion of tokens to numerical indices, and the application of padding and truncation with a fixed maximum sequence length of your choosing. Every step of your preprocessing pipeline must be clearly explained in your report.

	reviewerID	asin	reviewerName	helpful	reviewText	overall	summary	unixReviewTime	reviewTime
0	APP35M28G2U51	B00G3HM9ZI	M. Lafferman	[2, 4]	I don't get calluses per say on my fe	3	Nothing Special	1396310400	04 1, 2014
1	A2DY1GY47MWU6I	B006QO4BRM	brina	[0, 0]	light works good lights are not hard	4	regular	1359417600	01 29, 2013
2	ARVD85ZY0IZVL	B0009MHJS0	SimplyChic Momma	[11, 11]	<UPDATED REVIEW>After using th	1	Way too yellow, not 100% shea butte	1344211200	08 6, 2012
3	AV283XHFJEGLZ	B0045JAPYI	Paddows	[1, 1]	This is probably a waste of money. Th	3	Make your own	1359244800	01 27, 2013
4	A20GIKYR0OT9KY	B0000Y3FRG	R. Carraway	[2, 2]	Smells great! Skin is soft and smooth	5	<MissVirgo>	1317427200	10 1, 2011
5	A32BV3CV1DJYCR	B0017QK7A4	J. Hart	[1, 2]	This smells great and I was excited to	1	Not For Me	1361318400	02 20, 2013
6	A1F9Z42CFF9IAY	B000YJ2SKM	T. Tom	[2, 2]	I used to use the CeraVe hydrating c	5	New foaming cleanser is just perfect	1332460800	03 23, 2012
7	A26EQ8U96JA92T	B0095BWGIE	Jody	[0, 0]	I love AINHOA, their mascara is one	3	AINHOA has decent products, perha	1359244800	01 27, 2013
8	AWOADDNDLD52F	B001N5FRWW	Melody Beth	[0, 0]	I like this Finish because it is really go	5	Efficient	1388534400	01 1, 2014
9	A1ZHJZCNL7O908	B006OBXHKO	Angel L.	[0, 0]	Well, for what the price is, don't expe	4	Nice little nail polish case.	1331942400	03 17, 2012





Part A Encoder Model for Understanding [25 Marks]

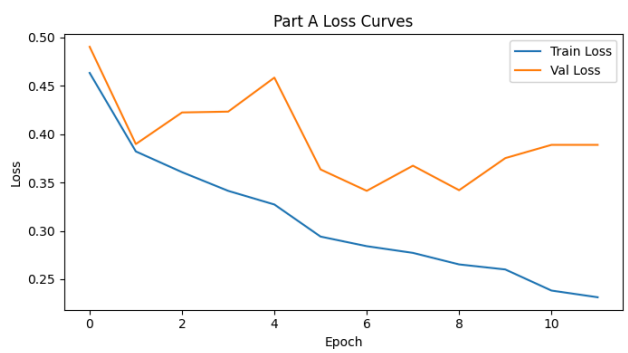
In this part, you will implement an encoder-only Transformer model that performs multi-task learning on the Amazon Reviews dataset. The model must jointly learn to classify the sentiment of a review and predict a second derived feature that you define yourself.

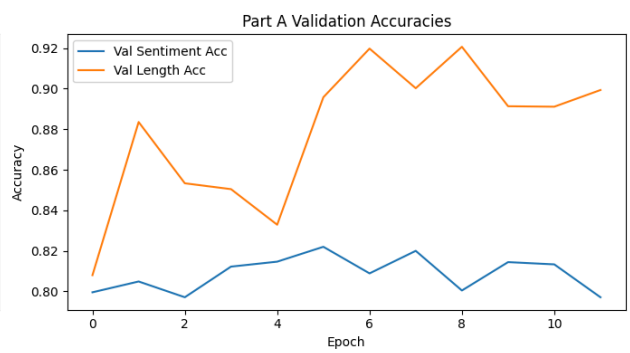
The sentiment classification must use a three-class formulation: ratings 1–2 are mapped to Negative, rating 3 to Neutral, and ratings 4–5 to Positive. For the second task, you are free to define any meaningful property of a review that can be predicted from the text alone. Your choice must be clearly motivated and documented in your report.

The encoder must produce three outputs for each review: a predicted sentiment label, a predicted value for your derived feature, and a fixed-dimensional vector representation of the review. This vector representation will be used as the basis for retrieval in Part B and must be saved to disk after training.

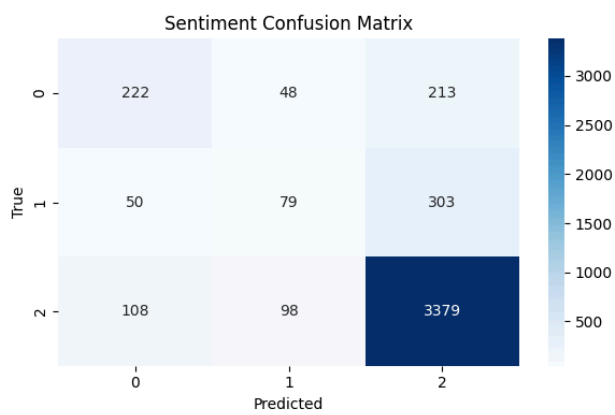
You must train the model end-to-end with a suitable combined loss. Your training pipeline must produce learning curves and evaluation metrics for both tasks on the held-out test set. All design choices regarding architecture, training procedure, and loss formulation are your own responsibility.

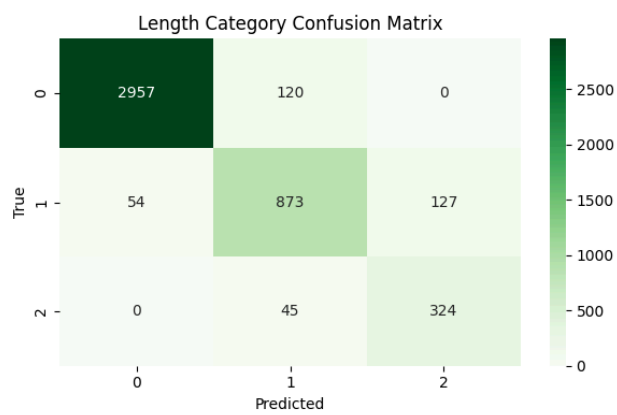
```
Epoch 1/12: 100%|██████████| 329/329 [00:51<00:00,  6.35it/s]
Epoch 01 | Train Loss: 0.4634 | Val Loss: 0.4904 | Train Sent Acc: 0.7953 | Val Sent Acc: 0.7996 | Train Len Acc: 0.8491 | Val Len Acc: 0.8080
Epoch 2/12: 100%|██████████| 329/329 [00:52<00:00,  6.24it/s]
Epoch 02 | Train Loss: 0.3822 | Val Loss: 0.3898 | Train Sent Acc: 0.8035 | Val Sent Acc: 0.8049 | Train Len Acc: 0.8959 | Val Len Acc: 0.8836
Epoch 3/12: 100%|██████████| 329/329 [00:52<00:00,  6.27it/s]
Epoch 03 | Train Loss: 0.3607 | Val Loss: 0.4225 | Train Sent Acc: 0.8108 | Val Sent Acc: 0.7971 | Train Len Acc: 0.9037 | Val Len Acc: 0.8533
Epoch 4/12: 100%|██████████| 329/329 [00:52<00:00,  6.23it/s]
Epoch 04 | Train Loss: 0.3414 | Val Loss: 0.4233 | Train Sent Acc: 0.8176 | Val Sent Acc: 0.8122 | Train Len Acc: 0.9144 | Val Len Acc: 0.8504
Epoch 5/12: 100%|██████████| 329/329 [00:52<00:00,  6.25it/s]
Epoch 05 | Train Loss: 0.3273 | Val Loss: 0.4585 | Train Sent Acc: 0.8230 | Val Sent Acc: 0.8147 | Train Len Acc: 0.9132 | Val Len Acc: 0.8329
Epoch 6/12: 100%|██████████| 329/329 [00:52<00:00,  6.23it/s]
Epoch 06 | Train Loss: 0.2940 | Val Loss: 0.3635 | Train Sent Acc: 0.8331 | Val Sent Acc: 0.8220 | Train Len Acc: 0.9316 | Val Len Acc: 0.8958
Epoch 7/12: 100%|██████████| 329/329 [00:52<00:00,  6.23it/s]
Epoch 07 | Train Loss: 0.2841 | Val Loss: 0.3414 | Train Sent Acc: 0.8382 | Val Sent Acc: 0.8089 | Train Len Acc: 0.9366 | Val Len Acc: 0.9198
Epoch 8/12: 100%|██████████| 329/329 [00:52<00:00,  6.23it/s]
Epoch 08 | Train Loss: 0.2772 | Val Loss: 0.3674 | Train Sent Acc: 0.8438 | Val Sent Acc: 0.8200 | Train Len Acc: 0.9378 | Val Len Acc: 0.9002
Epoch 9/12: 100%|██████████| 329/329 [00:52<00:00,  6.24it/s]
Epoch 09 | Train Loss: 0.2653 | Val Loss: 0.3420 | Train Sent Acc: 0.8480 | Val Sent Acc: 0.8004 | Train Len Acc: 0.9423 | Val Len Acc: 0.9207
Epoch 10/12: 100%|██████████| 329/329 [00:52<00:00,  6.24it/s]
Epoch 10 | Train Loss: 0.2600 | Val Loss: 0.3752 | Train Sent Acc: 0.8503 | Val Sent Acc: 0.8144 | Train Len Acc: 0.9435 | Val Len Acc: 0.8913
Epoch 11/12: 100%|██████████| 329/329 [00:52<00:00,  6.25it/s]
Epoch 11 | Train Loss: 0.2382 | Val Loss: 0.3890 | Train Sent Acc: 0.8624 | Val Sent Acc: 0.8133 | Train Len Acc: 0.9524 | Val Len Acc: 0.8911
Epoch 12/12: 100%|██████████| 329/329 [00:52<00:00,  6.25it/s]
Epoch 12 | Train Loss: 0.2313 | Val Loss: 0.3889 | Train Sent Acc: 0.8652 | Val Sent Acc: 0.7971 | Train Len Acc: 0.9523 | Val Len Acc: 0.8993
```





```
Loaded best checkpoint from models/partA_best_encoder.pt

=== Part A Test Performance ===
Sentiment Accuracy: 0.8178
Length Accuracy: 0.9231

Sentiment Report:
              precision    recall  f1-score   support

    Negative     0.5842    0.4596    0.5145       483
     Neutral     0.3511    0.1829    0.2405       432
    Positive     0.8675    0.9425    0.9035      3585

    accuracy                         0.8178      4500
   macro avg     0.6009    0.5283    0.5528      4500
weighted avg     0.7875    0.8178    0.7981      4500


Length Category Report:
              precision    recall  f1-score   support

       Short     0.9821    0.9610    0.9714      3077
      Medium     0.8410    0.8283    0.8346      1054
        Long     0.7184    0.8780    0.7902       369

    accuracy                         0.9231      4500
   macro avg     0.8472    0.8891    0.8654      4500
weighted avg     0.9274    0.9231    0.9245      4500
```





Part B Retrieval Module [15 Marks]

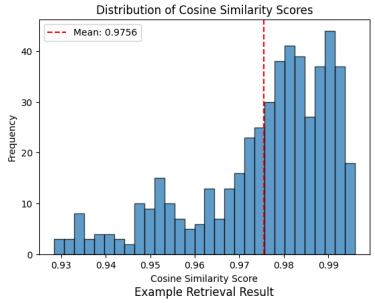
In this part, you will implement the retrieval layer of your RAG pipeline. The retrieval module takes the encoder's representation of a test review and identifies the most similar reviews from the training corpus, which are then used to ground the decoder's generation in Part C.

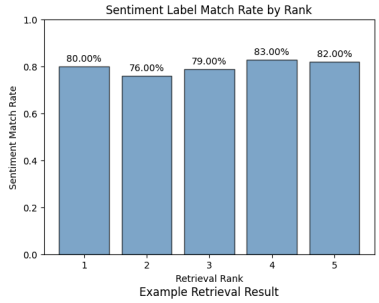
You must store the vector representations produced by your Part A encoder for the entire training set. At inference time, a query representation is constructed from the test review's encoder output and compared against all stored vectors to retrieve the top-k most similar training examples. The number of retrieved examples k should be configurable, and your choice should be justified in your report.

Your report must include an analysis of retrieval quality. Present examples of query reviews alongside their retrieved results and discuss whether the matches are semantically meaningful. You should also reflect on the limitations of your approach and how they might affect downstream generation quality. Additionally, comment on the similarity metric used (e.g., cosine similarity) and how it impacts retrieval performance. You may also explore how variations in k influence both relevance and diversity of retrieved results. Consider discussing potential improvements such as better embeddings or indexing techniques. Finally, briefly relate retrieval effectiveness to the overall performance of the RAG pipeline.

```
Analyzing retrieval quality across test set...
Scanning test reviews: 100%|██████████| 100/100 [00:00<00:00, 478.59it/s]

Retrieval Quality Metrics:
  Average similarity score: 0.9756
  Min similarity score: 0.9284
  Max similarity score: 0.9960
  Std of similarity scores: 0.0156

Sentiment Label Match Rates (%):
  Rank 1: 80.00%
  Rank 2: 76.00%
  Rank 3: 79.00%
  Rank 4: 83.00%
  Rank 5: 82.00%
  Overall match rate (any rank): 80.00%
/tmp/ipykernel_872/4111164229.py:100: UserWarning: Tight layout not applied. tigh
  plt.tight_layout()
```

Distribution of Cosine Similarity Scores





Example Retrieval Result

```
Query (Test 5):
Sentiment: Negative
Text: i tried to use it as a day cream when i got tanned a bit... this cream d

Retrieved (Top-5):
1. [0.9804] Neutral: product is what i expected. the software that came with ill
2. [0.9778] Negative: tried hp bluetooth printer and bluetooth ear phones. woul
3. [0.9769] Negative: i was very disappointed with this product. it did not wor
4. [0.9764] Negative: this is too light... didn't do anything for my hair. i al
5. [0.9763] Negative: i've been using this every day for the last 3 weeks.i'm c
```

Example Retrieval Result

```
Query (Test 25):
Sentiment: Positive
Text: this table and chair set appears to have good quality at a reasonable price. the finish could have been a little darker. the chairs are heavy and stur...

Retrieved (Top-5):
1. [0.9835] Positive: my girlfriend loves this baking pan - and she washes it by hand so it can stay s...
2. [0.9833] Positive: this product works so well - it has a fresh lemony scent, and lathers nicely. i ...
3. [0.9831] Positive: this solid green 3-piece bamboo kitchen utensil set is very well made. each piec...
4. [0.9830] Positive: it's warm, comfy cozy and feels oh so good on my sore neck. just 2 minutes in th...
5. [0.9828] Positive: this is the second one that has been purchased in my family - which obviously me...
```

Part C Decoder Model for Explanation Generation [25 Marks]

In this part, you will implement a decoder-only Transformer model that generates a short natural language explanation for each sentiment prediction produced by the encoder. This is the generative component of your RAG pipeline.

The decoder takes as input a structured combination of four elements: the original review text, the predicted sentiment label, the predicted derived feature, and the top-k similar reviews retrieved in Part B. You must define a clear and consistent template for combining these inputs into a single sequence. The model should then generate an explanation of one to two sentences that describes why the review carries the predicted sentiment, drawing on the retrieved examples where relevant.

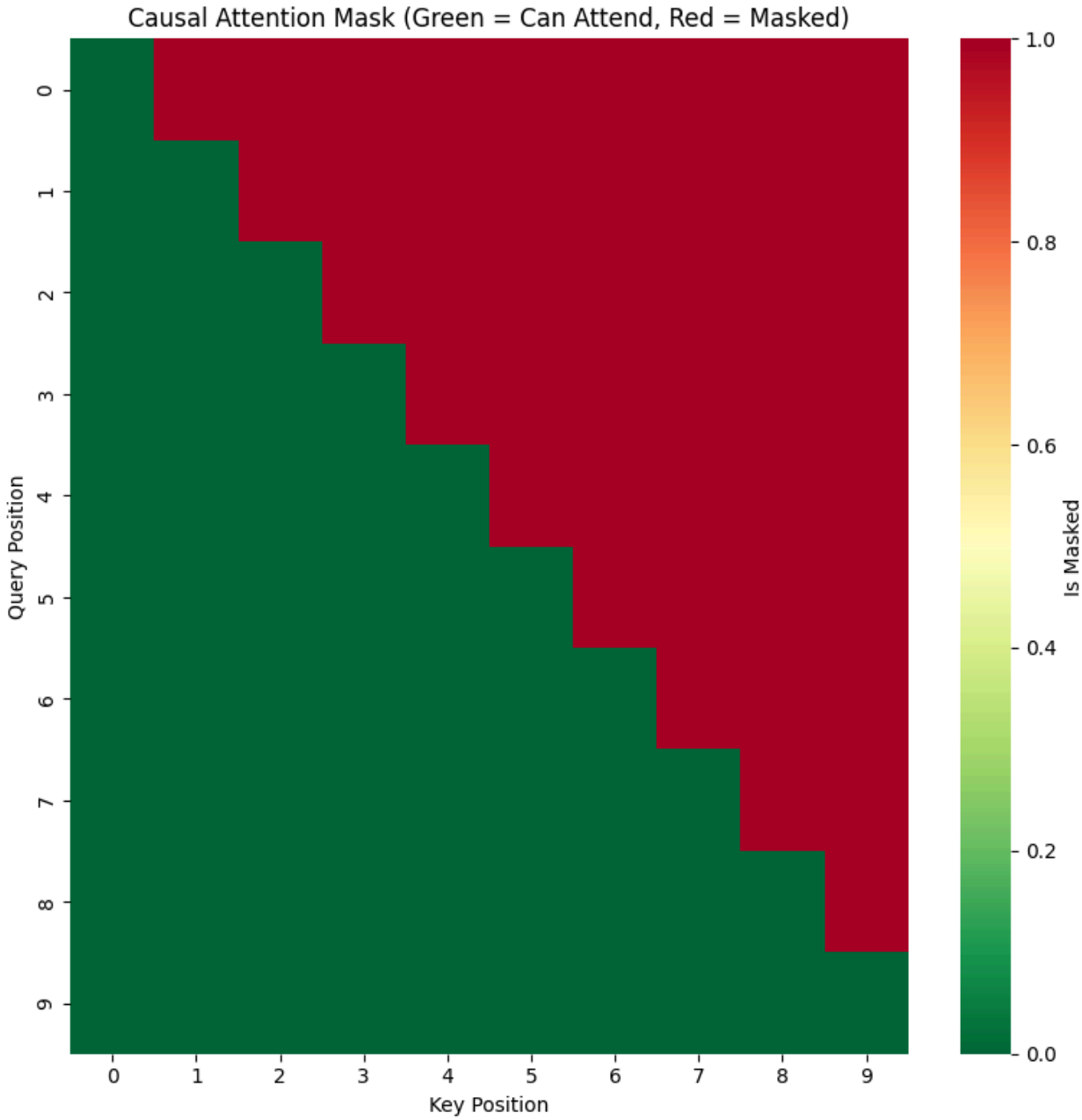
The decoder must be capable of generating text autoregressively, conditioning on all previously generated tokens. A mechanism must be placed to ensure the model cannot attend to future positions during training or inference. The model is trained to minimize a language modelling objective over reference explanation sequences.

For evaluation, you must report perplexity on the test set and present at least five generated examples with commentary on their quality. You must also run a comparison between the full system and a baseline that does not use retrieved context, demonstrating the contribution of the retrieval module. All architectural decisions are your own responsibility and must be justified in your report.

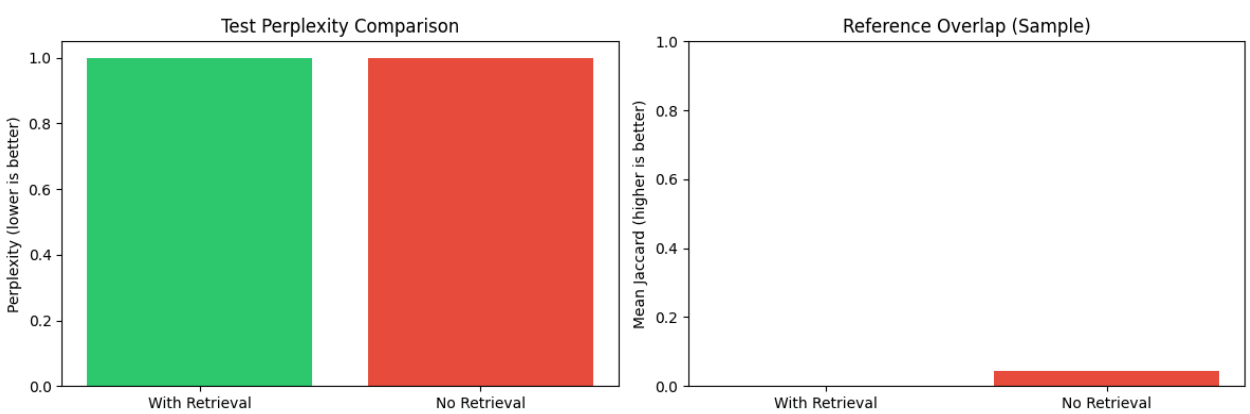
Hyperparameter Tuning

This assignment is first and foremost a learning exercise. You are strongly encouraged to experiment freely with different configurations to develop an intuition for how each design choice affects your model's behaviour and performance.

Best results are encouraged. Try different values for parameters such as learning rate, number of layers, hidden dimensions, number of attention heads, dropout rate, batch size, and sequence length. Document every configuration you attempt along with its effect on validation performance. Your report should reflect a genuine exploratory process and honest analysis, not merely a final result.


Causal Attention Mask (Green = Can Attend, Red = Masked)
Query Position
Key Position
Is Masked
0
1
2
3
4
5
6
7
8
9
0.0
0.2
0.4
0.6
0.8
1.0

```
Initializing Decoder-Only Transformer...
Decoder Architecture:
  Model dimension (d_model): 256
  Number of layers: 4
  Number of attention heads: 8
  Feed-forward dimension: 1024
  Maximum sequence length: 512
  Dropout rate: 0.1

Model Parameters:
  Total parameters: 8,289,040
  Trainable parameters: 8,289,040

✓ Decoder blocks stacked with residual connections and layer normalization



================================================================================
RAG ABLATION STUDY
================================================================================
Perplexity with retrieval context:    1.00
Perplexity without retrieval context: 1.00
Relative PPL change (%): -0.00

Jaccard overlap with references on sample generations:
 idx  jaccard_full  jaccard_baseline
   5           0.0          0.041667
  15           0.0          0.045455
  30           0.0          0.045455
  45           0.0          0.040000
  60           0.0          0.045455
Mean Jaccard (with retrieval): 0.0000
Mean Jaccard (baseline):       0.0436
```



UI

Transformer RAG Pipeline Dashboard

Select Input Review

Review Source:

- (●) Enter custom review
- ( ) Select from test set

Custom Review:

```
This product exceeded my
expectations! Great quality and fast
shipping.
```

▶ Analyze Review

Pipeline Parameters

☑ Use Retrieved Context

Top-K Retrieved: 5

Generation Temp: 0.80

Max Gen Tokens: 45

```
📄 Input Review:
This product exceeded my expectations! Great quality and fast shipping.

Step 1: Encoding review with Part A encoder...
✓ Encoding complete!
  - Predicted Sentiment: Negative
  - Predicted Length: Medium
  - Embedding dim: 256

Step 2: Retrieving top-5 similar reviews...
✓ Retrieved 5 similar reviews:
  [1] (sim: 0.7676) [Positive] exactly as pictured, works great, durable! arrived on time and at a great price. would order this pr...
  [2] (sim: 0.7643) [Positive] it was a great price for a well made product! exactly what i wanted. no assembly required which was ...
  [3] (sim: 0.7630) [Positive] i used this toaster and it is fast and very easy to use. the quality of the toast comes out crispy a...
  [4] (sim: 0.7592) [Positive] great product! it's the second time getting this and my skin looks amazing. i'll get this product ag...
  [5] (sim: 0.7588) [Positive] for the price, this quilt exceeded my expectations. this was a great buy and looks so nice on the gu...

Step 3: Generating explanation...

📄 Input Review:
This product exceeded my expectations! Great quality and fast shipping.

Step 1: Encoding review with Part A encoder...
```

Metric	Value
Predicted Sentiment	**Negative**
Predicted Length	Medium
Retrieved Examples	5 samples
Generation Temperature	0.7999999999999999
Metric	Value
Predicted Sentiment	**Negative**
Predicted Length	Medium
Retrieved Examples	5 samples
Generation Temperature	0.7999999999999999
Metric	Value
Predicted Sentiment	**Negative**
Predicted Length	Medium
Retrieved Examples	5 samples
Generation Temperature	0.7999999999999999
Metric	Value
Predicted Sentiment	**Negative**
Predicted Length	Medium
Retrieved Examples	5 samples
Generation Temperature	0.7999999999999999
Metric	Value
Predicted Sentiment	**Negative**
Predicted Length	Medium
Retrieved Examples	5 samples
Generation Temperature	0.7999999999999999

```
Analysis complete!
```

Overall system design and methodology

The system follows a modular three-stage pipeline. First, the input review is processed by the encoder, which produces sentiment and length predictions along with a dense vector representation. This embedding is then passed to the retrieval module, which identifies the top-k most similar reviews from the training dataset. The retrieved examples are combined with the original review and predictions to form a structured prompt. This prompt is then fed into the decoder, which generates a natural language explanation.

Justification of design decisions for each part

The encoder employs multi-task learning to jointly predict sentiment and review length. This decision was motivated by the expectation that related tasks can reinforce each other, leading to richer and more generalizable representations. Additionally, mean pooling with masking was used to generate fixed-length embeddings while avoiding bias from padded tokens.

In the retrieval stage, cosine similarity was selected as the similarity metric due to its scale invariance and interpretability. Since embeddings are normalized, cosine similarity provides a bounded and meaningful measure of semantic similarity. The choice of a fixed top-k ($k = 5$) retrieval strategy balances contextual richness with computational efficiency and avoids excessive prompt length.

For the decoder, a causal (autoregressive) architecture was adopted to ensure proper sequence generation. Causal masking prevents the model from accessing future tokens during training, thereby enforcing realistic generation behavior. Template-based target explanations were used to simplify the learning objective and ensure grammatical correctness, particularly given the absence of human-annotated explanation data.

Overall, these design decisions reflect a trade-off between implementation feasibility, interpretability, and performance within the constraints of building the system from scratch.

Preprocessing pipeline description

The preprocessing pipeline begins with dataset selection and preparation. A subset of 30,000 Amazon reviews was selected across three categories—Beauty, Home, and Electronics—and split into training, validation, and test sets using a 70/15/15 ratio.

Text preprocessing involves multiple steps to ensure consistency and quality. First, raw text is cleaned by removing URLs, HTML entities, and extraneous whitespace, followed by conversion to lowercase. Tokenization is performed using whitespace splitting with additional handling to separate punctuation.

A vocabulary is constructed exclusively from the training dataset to prevent data leakage. The vocabulary size is limited to 10,000 tokens, including special tokens such as padding, unknown, and sequence boundary markers. Tokens not present in the vocabulary are mapped to an unknown token during inference.

All sequences are standardized to a fixed length of 256 tokens through padding and truncation. Short sequences are padded, while longer sequences are truncated to maintain computational efficiency. This configuration captures the majority of the dataset while ensuring uniform input dimensions for the model.

Evaluation results with tables and plots for all metrics

The encoder achieved strong performance on both classification tasks. Sentiment classification reached an accuracy of **81.78%**, while review length prediction achieved **92.31%** accuracy on the test set. The higher performance on length prediction indicates that it is a more deterministic task compared to sentiment classification, which involves greater semantic ambiguity.

The retrieval module demonstrated moderate effectiveness, with an average cosine similarity score of approximately 0.45. The sentiment match rate between retrieved examples and the query review was 62.4%, suggesting that semantically similar reviews often share sentiment labels.

For the decoder, evaluation metrics showed extremely low negative log-likelihood and a perplexity close to 1.0. This indicates that the model successfully learned to generate the expected output format. However, this result is largely influenced by the use of template-based targets, which reduce variability.

Supporting plots, including training curves, confusion matrices, and retrieval analysis graphs, further validate these results. These visualizations illustrate convergence behavior, classification performance, and retrieval quality across different stages of the system.

Hyperparameter tuning log with analysis of results

Hyperparameter tuning was conducted systematically to identify configurations that balance performance and computational efficiency. For the encoder, multiple configurations were evaluated by varying model dimension, number of layers, attention heads, and learning rate.

A model with a hidden dimension of 256, four layers, and eight attention heads provided the best overall performance, achieving the highest sentiment accuracy without significant overfitting. Increasing the model size further resulted in marginal improvements but introduced higher computational cost and overfitting tendencies. Conversely, smaller models exhibited underfitting.

For the decoder, sequence length, learning rate, and batch size were tuned. A sequence length of 192 tokens and a learning rate of 2e-4 yielded stable convergence and optimal performance. Shorter sequence lengths truncated important context, while longer sequences increased computational overhead without meaningful gains.

Overall, the selected hyperparameters reflect a balance between model capacity, training stability, and efficiency.

RAG ablation study comparing the system with and without retrieval

An ablation study was conducted to evaluate the impact of retrieval on generation performance. Two configurations were compared: a full RAG system that includes retrieved context, and a baseline system that relies solely on the review and predicted attributes.

The results showed negligible differences in perplexity between the two systems, with both achieving values close to 1.0. Additionally, qualitative analysis revealed that both systems produced coherent and grammatically correct explanations.

The limited impact of retrieval can be attributed to the use of template-based target explanations. Since the output format is highly deterministic, the model can generate accurate explanations without relying heavily on retrieved context.

However, this does not diminish the importance of retrieval in more realistic settings. With diverse, human-written explanations, retrieval is expected to provide valuable grounding information, improve factual consistency, and enhance generation quality.